

Proyecto: Sistema de Registro de Ventas en una Tienda

Introducción a la Programación

Grupo #3

Autores:

Fabian A. Gutierrez, Sebastián González Salas, Jeremy Valerio Meza

Universidad Fidelitas.

Jonathan Quesada Campos

I cuatrimestre

Índice

I. Antecedentes.....	3
II. Justificación.....	4
III. Diseño Preliminar	6
IV. Algoritmo	11
I. Diagrama de Flujo.....	23
II. Bibliografías	30

I. Antecedentes

En la era digital, los sistemas informáticos de registro de ventas se han vuelto esenciales para la administración eficiente de tiendas y comercios de todo tipo. Estos sistemas permiten registrar productos, calcular automáticamente los totales con sus respectivos impuestos y descuentos, y generar reportes útiles que apoyan la toma de decisiones. Según Laudon y Laudon (2020), “los sistemas de información mejoran significativamente la eficiencia operativa y permiten a las organizaciones responder rápidamente a los cambios del entorno” (p. 34), lo cual es vital en un entorno comercial dinámico.

La implementación de este tipo de sistemas también tiene un impacto directo en la sociedad, ya que fomenta la formalización de pequeñas empresas y facilita su crecimiento. De acuerdo con el Instituto Nacional de Estadística y Censos (INEC, 2021), más del 80% de los comercios minoristas en países en desarrollo aún gestionan sus ventas manualmente, lo que limita su eficiencia y capacidad de expansión. Por lo tanto, un sistema automatizado de ventas puede representar una solución real a un problema común.

Desde el ámbito educativo, este proyecto promueve la aplicación de conocimientos adquiridos durante el curso, tales como lógica de programación, estructuras de datos, diseño de interfaces gráficas, y bases de datos. Como señalan Pressman y Maxim (2020), el desarrollo de software no solo implica programación, sino también “el análisis de requerimientos, el diseño estructurado, la validación y el mantenimiento del sistema” (p. 12), lo cual se ve reflejado en el trabajo práctico que representa este proyecto.

Asimismo, este tipo de proyecto prepara al estudiante para enfrentar retos profesionales reales. En el entorno laboral, el desarrollo de sistemas personalizados es una competencia muy demandada. En palabras de Sommerville (2016), “la ingeniería de software debe formar parte del proceso de resolución de problemas del mundo real” (p. 6), lo cual se logra a través de proyectos aplicados como este.

En resumen, este sistema no solo responde a una necesidad común en los comercios modernos, sino que también representa una oportunidad educativa y profesional para aplicar conocimientos técnicos a una situación real del entorno económico y social.

II. Justificación

Una programación diseñada para automatizar las ventas del Abastecedor El Roble donde se logra ver la notable importancia de la tecnología en el comercio actual. La automatización de procesos no solo mejora cómo operan los negocios que también reducen los potenciales errores, además que también permite una gestión más exacta, disminuyendo errores humanos y mejorando decisiones. Abidemi (2024) señala que los sistemas tecnológicos en pequeñas y medianas empresas (pymes) han mostrado un aumento significativo en la productividad y una reducción del margen de error en tareas repetitivas.

El sistema tiene tres partes clave: el módulo de caja, el registro de ventas y el módulo de informes. Cada una de estas partes optimiza una necesidad específica de procesos comerciales modernos, haciéndolos más fácil para la organización, el análisis de datos y la experiencia del cliente.

El módulo de caja permite registrar productos, aplicar descuentos, calcular impuestos y emitir facturas de forma rápida y precisa. Esto significa una atención al cliente más ágil y profesional, algo vital en mercados muy competitivos. Según Nájera Ruiz y Collazzo (2021), el uso de sistemas de contabilidad y facturación en microempresas mejora el control de las operaciones y promueve la transparencia financiera.

El registro de ventas proporciona un registro atento de cada operación efectuada. Este módulo no solo simplifica el monitoreo de las operaciones, sino que además favorece una administración más responsable y eficaz de las otras. Abidemi (2024) indica que este tipo de automatización ayuda a las pymes a reducir hasta un 25% los errores administrativos relacionados con la entrada manual de datos, lo que aumenta la confianza en la información usada para tomar decisiones.

Finalmente, el módulo de informes ofrece herramientas de análisis que permiten observar los datos importantes recopilados y como el total de ventas de cada producto también el identificar los artículos más vendidos y generando reportes diarios. La capacidad de analizar tendencias de venta y ajustar estrategias basándose en datos reales coincide con la de su demanda en esta era de digitalización que proponen Nájera Ruiz y Collazzo (2021). Ellos afirman que los sistemas digitales permiten a las microempresas responder mejor a los cambios del mercado.

III. Diseño Preliminar

Para efectos de la creación del programa, se busca centrarlo en 3 módulos distintos que le permitan a un usuario realizar diferentes acciones; los cuales lo enfocaremos en un registro de ventas en una tienda el cual le permita a la misma agilizar los procesos para la compra y venta de productos.

Esta va a tener como productos a la venta los siguientes: Bebida, banano, sandia, galleta, mantequilla, leche, lechuga, tomate y alcohol

Para el menú de entrada se busca el siguiente:

----ABASTECEDOR EL ROBLE----

1. Cajero
2. Registro y visualización de cada venta
3. Módulo de informes

Con estas opciones, se espera que el usuario ingrese uno de los números de entre el 1 al 4 para acceder a la opción que se requiere en ese momento.

Para la primera opción, se busca que el usuario ingrese un producto en conjunto de su cantidad y su precio para cuando el usuario no agregue más productos, este de los totales y descuentos específicos. Este se varia de la siguiente manera:

----Cajero-----

Lista de precios

Bebida = 800

Banano = 250

Sandia = 2000

Galleta = 550

Mantequilla = 1250

Leche = 1100

Lechuga = 6000

Tomate = 300

Alcohol = 1000

Ingrese su producto:

Cuanta cantidad desea comprar:

¿Desea agregar otro producto? (S = Si, N = No):

(En caso de descuento)

¿Tienes algún descuento? (S =Si, N = No)

Ingrese su descuento (N1=3%, N2=5%, N3=7%):

---Totales----

Venta #1:

Productos:

Total bruto:

Impuesto por IVA (13%):

Descuento:

Total neto:

Posteriormente para el menú de "Registro y visualización de cada venta" aparecerá lo siguiente:

Venta #1:

Productos:

Total bruto:

Impuesto por IVA (13%):

Descuento:

Total neto:

Venta #2:

Productos:

Total bruto:

Impuesto por IVA (13%):

Descuento:

Total neto:

Venta #3:

Productos:

Total bruto:

Impuesto por IVA (13%):

Descuento:

Total neto:

Cada venta que se haga, el registro se quedara guardado con el fin de que cuando se coloque esta opción, de todas las ventas hechas en el día.

Para el "Modulo de informes", cuando se seleccione esta opción, se desplegará otro menú el cual pedirá lo siguiente:

1. Total de ventas por producto.
2. Producto más vendido del día.

Estos desplegarían los siguientes datos:

-----Total de ventas por producto-----

Bebida = total_Bebida

Banano = total_Banano

Sandia = total_Sandia

Galleta = total_Galleta

Mantequilla = total_Mantequilla

Leche = total_Leche

Lechuga = total_Lechuga

Tomate = total_Tomate

Alcohol = total_Alcohol

----Producto mas vendido del dia-----

producto = product

Con este acomodo, es que se va a visualizar el programa a la hora de ejecutarlo, con el ideal de que la persona que haga uso del programa, tenga los datos detallados y de facil alcance.

IV. Algoritmo

Algoritmo del módulo “main.py” del Sistema de Registro de Ventas

Inicio del algoritmo

Iniciar el sistema mostrando el menú principal con cuatro opciones:

1. Cajero
2. Registro y visualización de cada venta
3. Módulo de informes
4. Salida del sistema

Crear una lista vacía llamada sesiones que servirá para guardar todas las sesiones de ventas registradas durante el uso del sistema.

Iniciar un bucle de repetición (while) infinito que se mantendrá activo hasta que el usuario elija salir del sistema.

Dentro del bucle, solicitar al usuario que ingrese una opción del menú principal (guardada en la variable opcion_cajero).

Si la opción ingresada es 1 (Cajero):

Iniciar una nueva sesión de ventas, creando una lista vacía llamada ventas para guardar los productos comprados.

Iniciar un segundo bucle (while) para registrar cada venta dentro de la sesión.

En cada iteración:

Solicitar al usuario que ingrese el nombre de un producto.

Convertirlo en formato capitalizado (por ejemplo, “pan” → “Pan”).

Verificar si ese producto existe en el diccionario productos.

Si el producto existe:

Obtener su precio desde el diccionario.

Solicitar la cantidad que desea llevar.

Llamar al subprograma registrar_venta(producto, precio, cantidad) para registrar la venta.

Agregar la venta a la lista ventas.

Si el producto no existe:

Mostrar un mensaje de error indicando que no es válido.

Preguntar si el usuario desea salir del sistema de cajero (responder con "S" para salir).

Si el usuario decide salir y la lista ventas tiene al menos un producto:

Agregar la lista ventas a la lista general sesiones.

Estructuras utilizadas:

Repetición: while para registrar varias ventas en una sesión.

Decisión: if producto in productos, if salida_cajero == "S".

Subprograma: registrar_venta(...).

Variables utilizadas:

producto (nombre del producto)

precio (precio individual)

cantidad (cantidad del producto)

venta (registro individual de una venta como diccionario)

ventas (lista de ventas por sesión)

sesiones (lista global de todas las sesiones)

Cálculos:

Multipliación: $\text{precio} * \text{cantidad}$ (realizado en registrar_venta)

Si la opción ingresada es 2 (Registro y visualización de ventas):

Verificar si hay alguna venta registrada en la lista sesiones.

Si no hay ninguna sesión:

Mostrar mensaje de que no existen ventas.

Si hay sesiones:

Recorrer la lista sesiones utilizando un bucle for, enumerando cada sesión con su número.

Para cada sesión:

Mostrar cada venta usando el subprograma `mostrarunaventa(venta, ventas)`.

Calcular el total de la sesión sumando el producto de precio * cantidad para cada venta en la sesión.

Mostrar el total en pantalla.

Estructuras utilizadas:

Decisión: `if not sesiones`

Repetición: `for idx, ventas in enumerate(...)`

Subprograma: `mostrarunaventa(...)`

Variables utilizadas:

`ventas, venta, total_sesion, idx`

Cálculos:

Suma de `precio * cantidad` usando una expresión con `sum(...)`.

Si la opción ingresada es 3 (Módulo de informes):

Solicitar al usuario que ingrese el nombre de un producto para generar un informe.

Verificar si el producto existe en el diccionario `productos`.

Si es válido:

Obtener su precio.

Crear una lista unificada de ventas llamada `todas_las_ventas`, que combina todas las ventas de todas las sesiones.

Llamar al subprograma `generar_informe(producto, todas_las_ventas, precio, 0)`.

Si no es válido:

Mostrar un mensaje de error.

Estructuras utilizadas:

Decisión: `if producto in productos`

Subprograma: `generar_informe(...)`

Variables utilizadas:

`producto, precio, todas_las_ventas`

Si la opción ingresada es 4 (Salir):

Mostrar un mensaje de despedida.

Finalizar el programa con `break`.

Si se ingresa una opción no válida:

Mostrar mensaje de opción incorrecta y salir con `break`.

Fin del algoritmo

El sistema ha terminado correctamente.

Algoritmo del módulo “productos.py” del Sistema de Registro de Ventas

Inicio del algoritmo

Se define una estructura de datos tipo diccionario llamada productos.

Este diccionario contiene una lista de productos disponibles en la tienda.

Cada producto está asociado a su precio en colones costarricenses (₡).

Contenido del diccionario productos:

Producto	Precio (₡)
Bebida	800
Banano	250
Sandia	2000
Galleta	550
Mantequilla	1250
Leche	1100
Lechuga	6000
Tomate	300
Alcohol	1000

Descripción detallada en lenguaje natural:

Este módulo se encarga únicamente de almacenar y proporcionar los productos disponibles para la venta en el sistema principal (main.py).

Cada vez que el usuario ingresa el nombre de un producto, el sistema consulta si el nombre ingresado existe como clave en este diccionario.

Si el producto está presente:

El sistema puede recuperar su precio para realizar el cálculo total de la venta (precio * cantidad).

Si el producto no está presente:

El sistema mostrará un mensaje de error indicando que el producto es inválido.

Estructuras utilizadas:

Estructura de datos: Diccionario (productos)

Clave = Nombre del producto (tipo str)

Valor = Precio del producto (tipo int)

Variables utilizadas:

productos: Diccionario que almacena toda la información sobre los productos disponibles.

Claves como "Bebida", "Banano", etc.

Valores numéricos enteros como 800, 250, 2000, etc.

Cálculos realizados:

No se realizan cálculos directamente en este archivo.

Los precios definidos aquí son usados posteriormente para calcular:

El total por producto (precio * cantidad)

El total de la sesión

Los informes generales

Fin del algoritmo

Este módulo se importa en el archivo principal y permite el acceso a los datos base del sistema de ventas.

Algoritmo del módulo “funcionestrabajo.py” del Sistema de Registro de Ventas

Inicio del algoritmo

Este módulo contiene tres subprogramas (funciones) que permiten registrar, mostrar y generar informes sobre las ventas realizadas en la tienda.

Cada función está escrita en lenguaje Python y trabaja de forma modular para apoyar al archivo principal (main.py).

1. Subprograma registrar_venta(producto, precio, cantidad)

▪ Descripción:

Recibe tres datos como parámetros:

producto: nombre del producto vendido.

precio: precio unitario del producto.

cantidad: cantidad de unidades compradas.

Crea un diccionario llamado venta que contiene estos tres valores.

Retorna ese diccionario al sistema principal para ser almacenado en la lista de ventas de la sesión.

▪ **Estructuras utilizadas:**

Subprograma (función)

Diccionario

▪ **Variables utilizadas:**

producto, precio, cantidad (parámetros)

venta: diccionario con los datos de una venta

▪ **Cálculos:**

No realiza cálculos, solo empaqueta datos.

2. Subprograma mostrarunaventa(venta, ventas_sesion)

▪ **Descripción:**

Recibe:

venta: un diccionario que contiene la información de un producto vendido.

ventas_sesion: lista de ventas de una sesión (no se usa activamente en esta función, pero se pasa como parámetro).

Extrae del diccionario:

producto: nombre del producto.

precio: precio unitario.

cantidad: cantidad comprada.

Calcula el total de esa venta individual multiplicando $\text{precio} * \text{cantidad}$.

Muestra en pantalla el detalle del producto comprado y su total.

▪ Estructuras utilizadas:

Subprograma (función)

Diccionario

Cálculo aritmético ($\text{precio} * \text{cantidad}$)

▪ Variables utilizadas:

producto, precio, cantidad, total

▪ Cálculos:

Multiplicación: $\text{total} = \text{precio} * \text{cantidad}$

3. Subprograma generar_informe(producto, ventas, precio, cantidad)

▪ Descripción:

Recibe:

producto: nombre del producto sobre el cual se va a generar el informe.

ventas: una lista con todas las ventas realizadas en todas las sesiones.

precio y cantidad: aunque se reciben como parámetros, no se usan directamente dentro de esta función.

Inicializa dos variables:

total_general: para sumar el total en colones del producto solicitado.

total_cantidad: para contar cuántas unidades se vendieron del producto.

Recorre cada venta de la lista ventas con un bucle for.

En cada iteración:

Verifica si el producto de la venta coincide con el producto solicitado.

Si coincide:

Suma al total_general el valor de esa venta (precio * cantidad).

Suma al total_cantidad la cantidad vendida.

Al finalizar el ciclo, muestra:

Cuántas unidades del producto se vendieron.

Cuánto dinero se generó en total por ese producto.

▪ Estructuras utilizadas:

Subprograma (función)

Repetición (for)

Decisión (if)

Diccionario

▪ Variables utilizadas:

producto, ventas, precio, cantidad (parámetros)

total_general, total_cantidad

i (cada venta en la lista)

▪ Cálculos:

$\text{total_general} += \text{precio} * \text{cantidad}$

$\text{total_cantidad} += \text{cantidad}$

Fin del algoritmo

Este módulo complementa al sistema de ventas permitiendo:

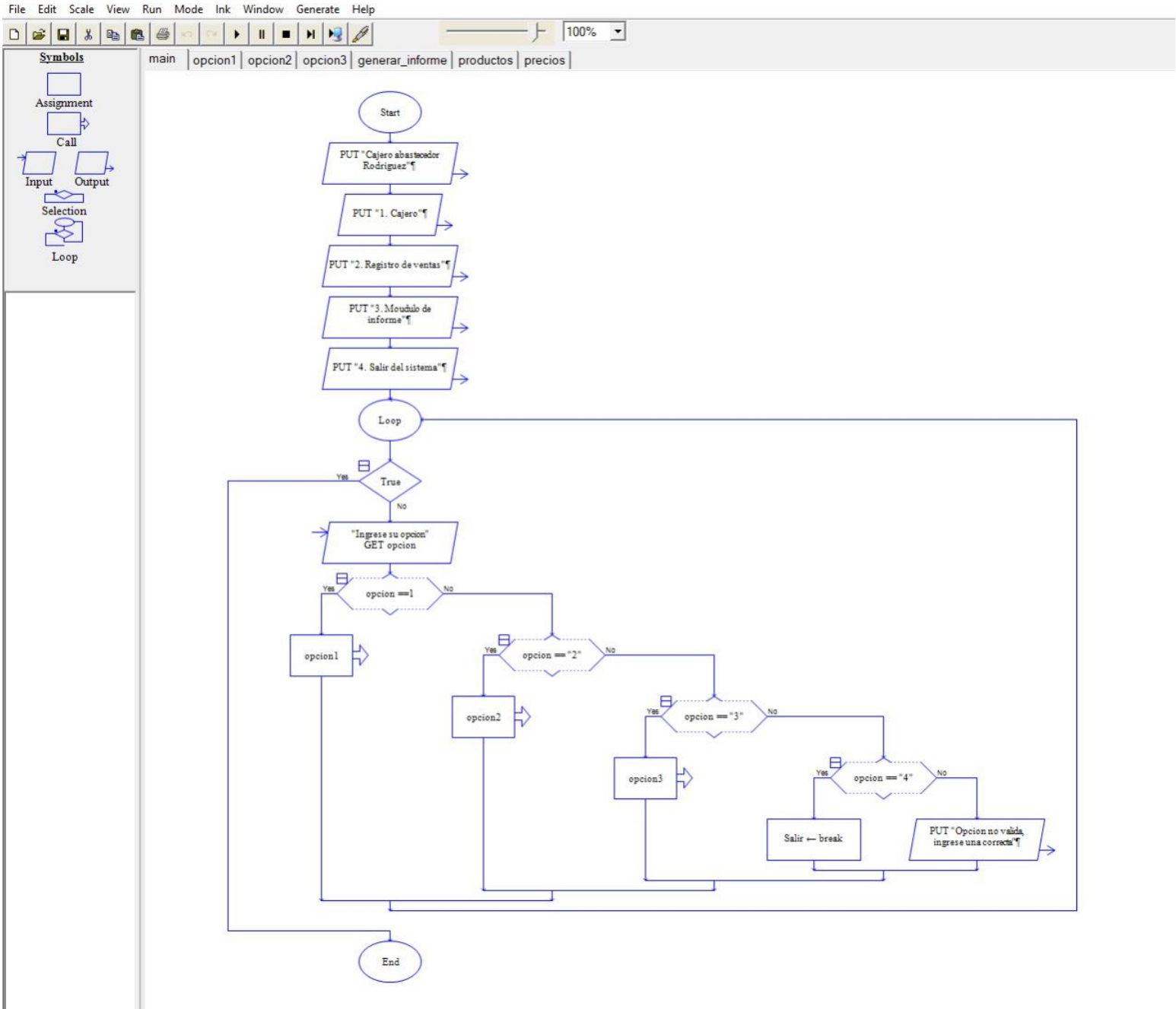
Registrar ventas de manera organizada.

Mostrar al usuario los detalles de cada producto vendido.

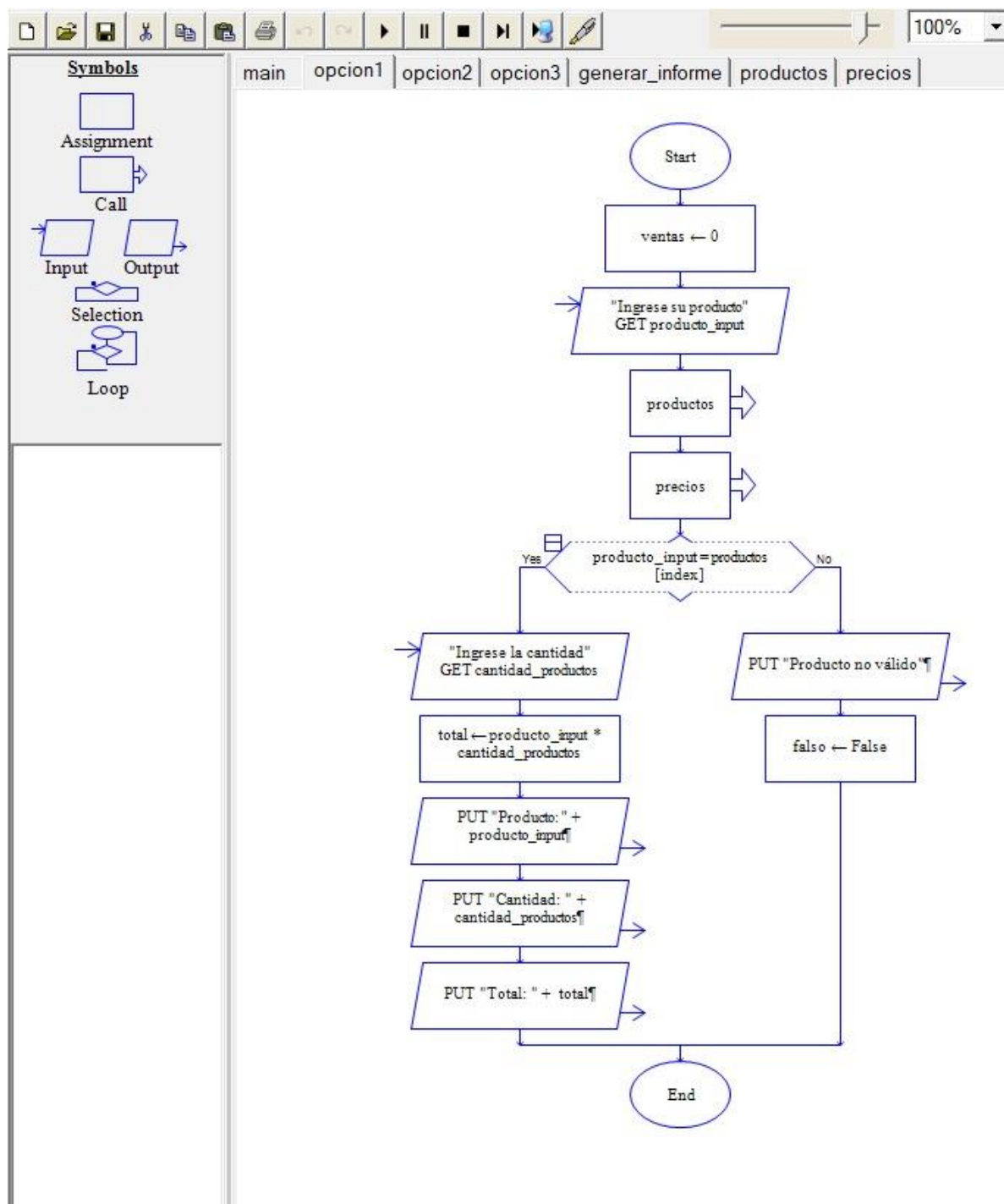
Generar informes útiles sobre el comportamiento de ventas por producto.

I. Diagrama de Flujo

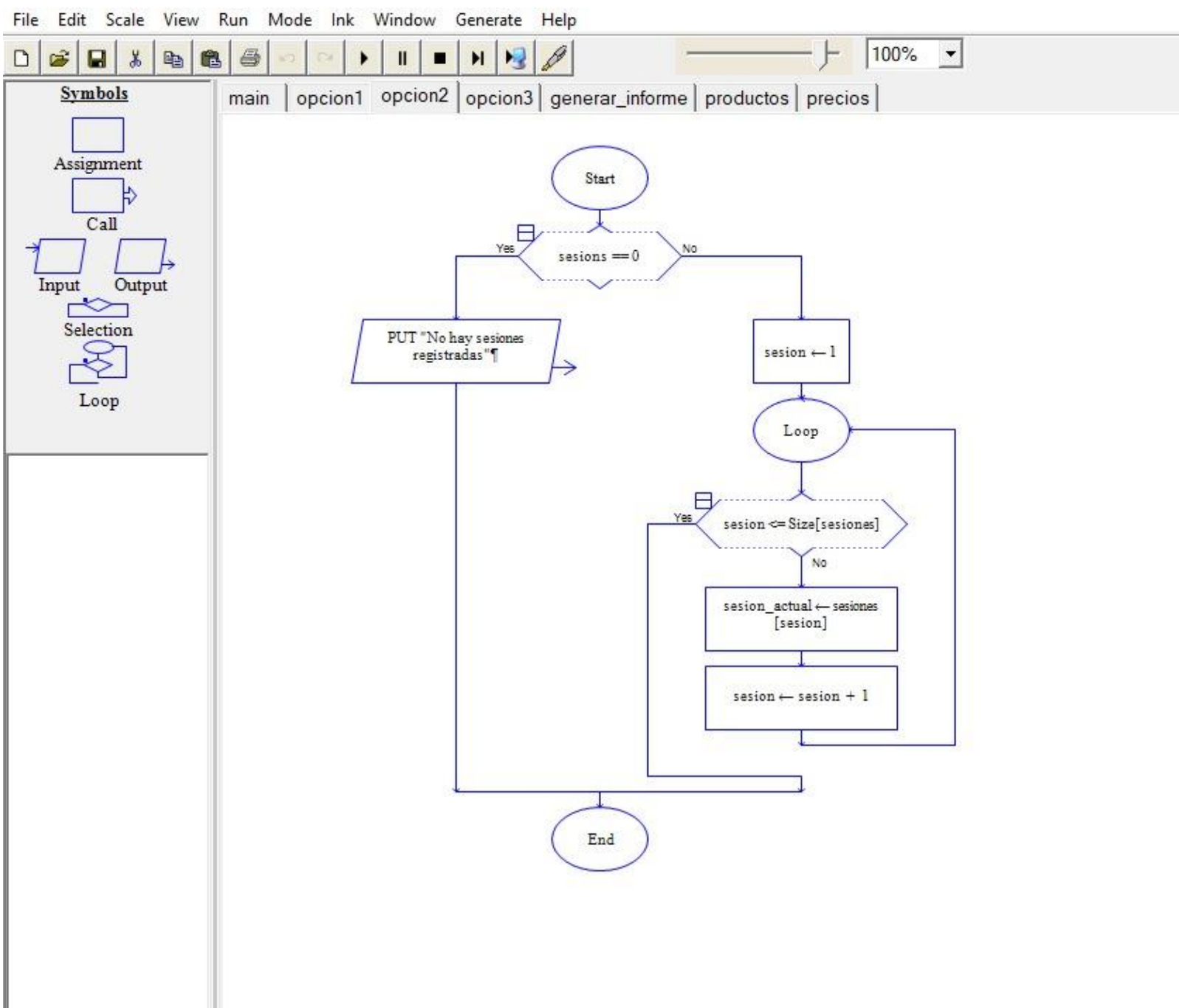
- Main:



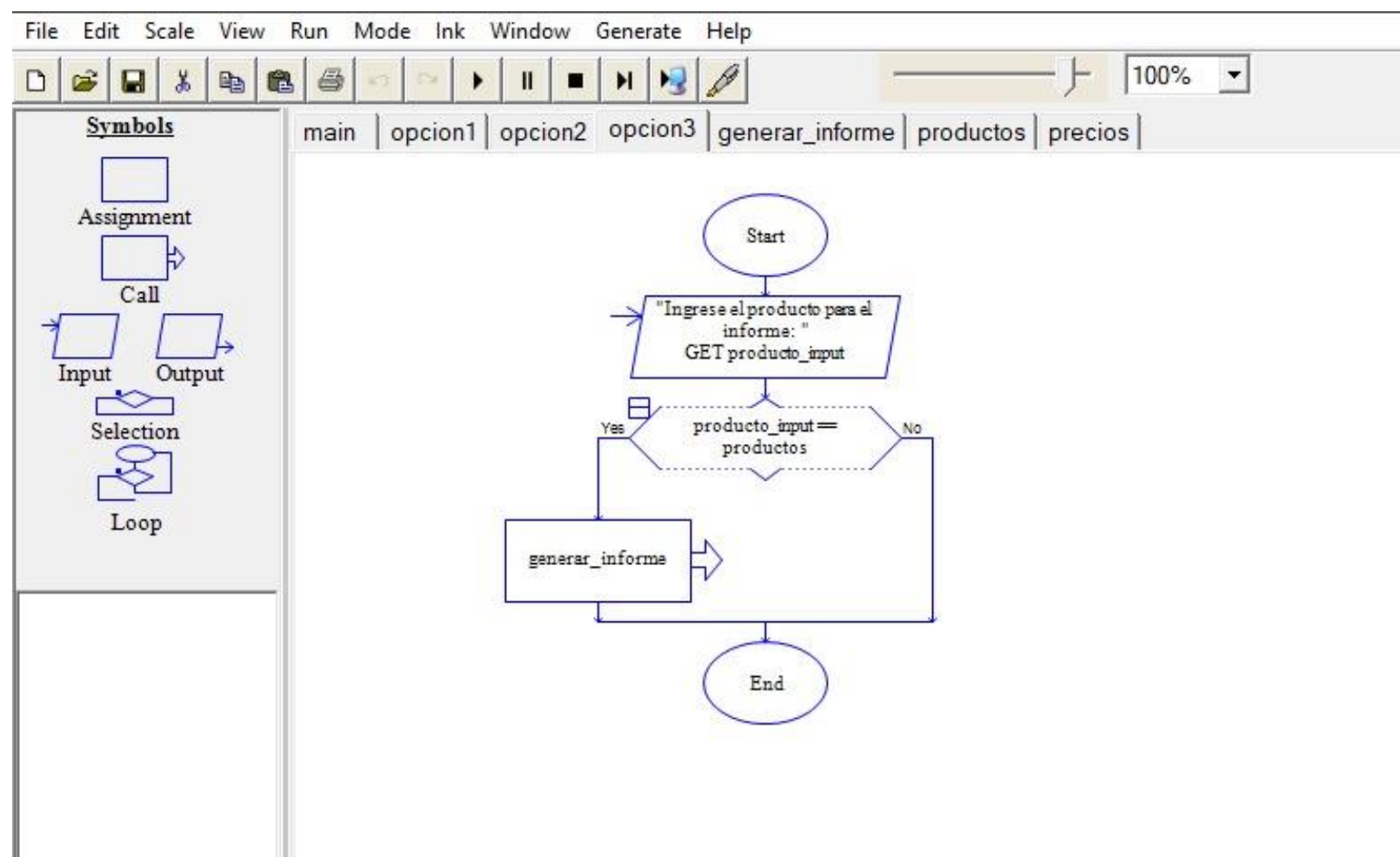
- Opción 1:



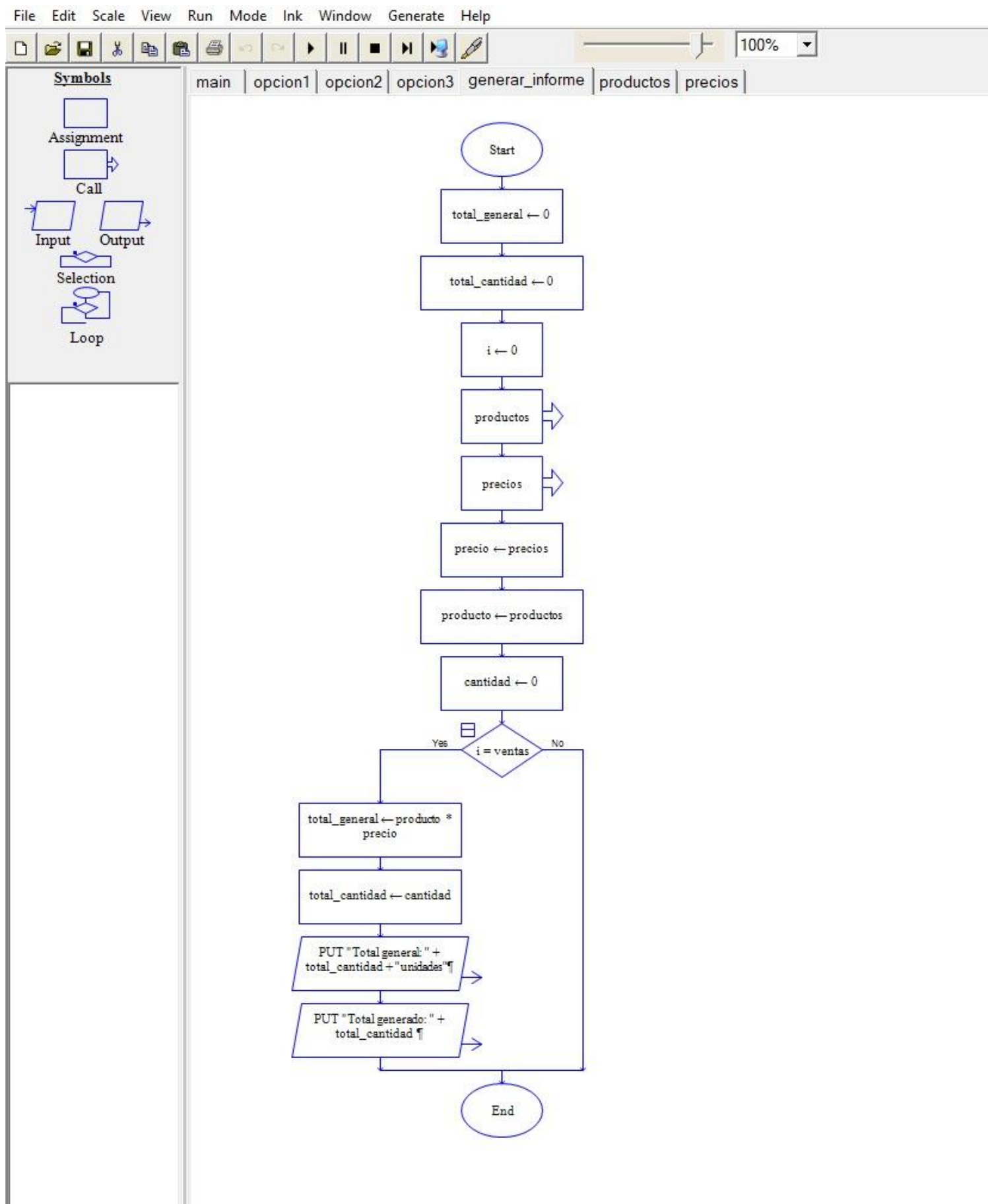
- Opción 2:



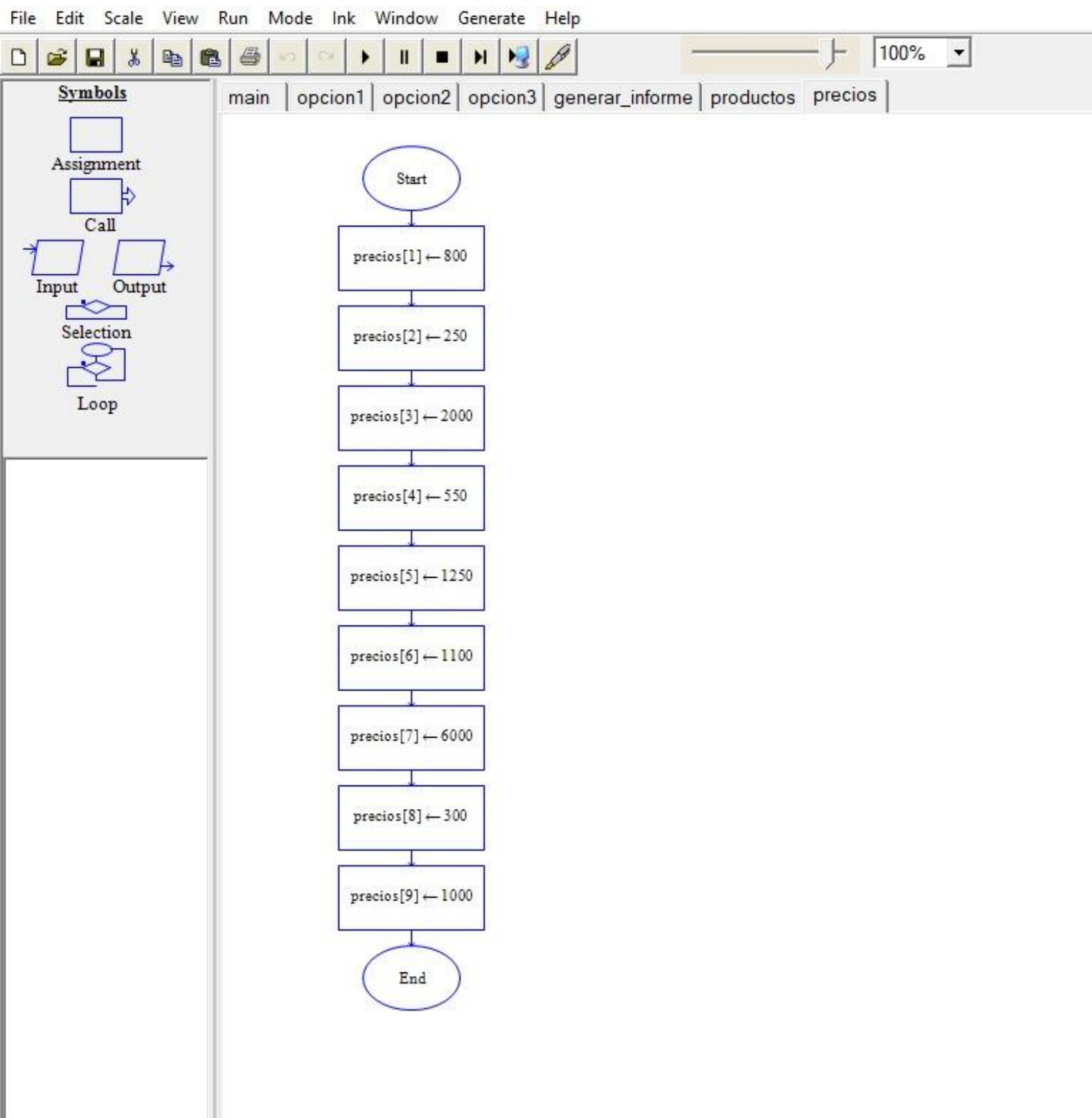
- Opción 3:



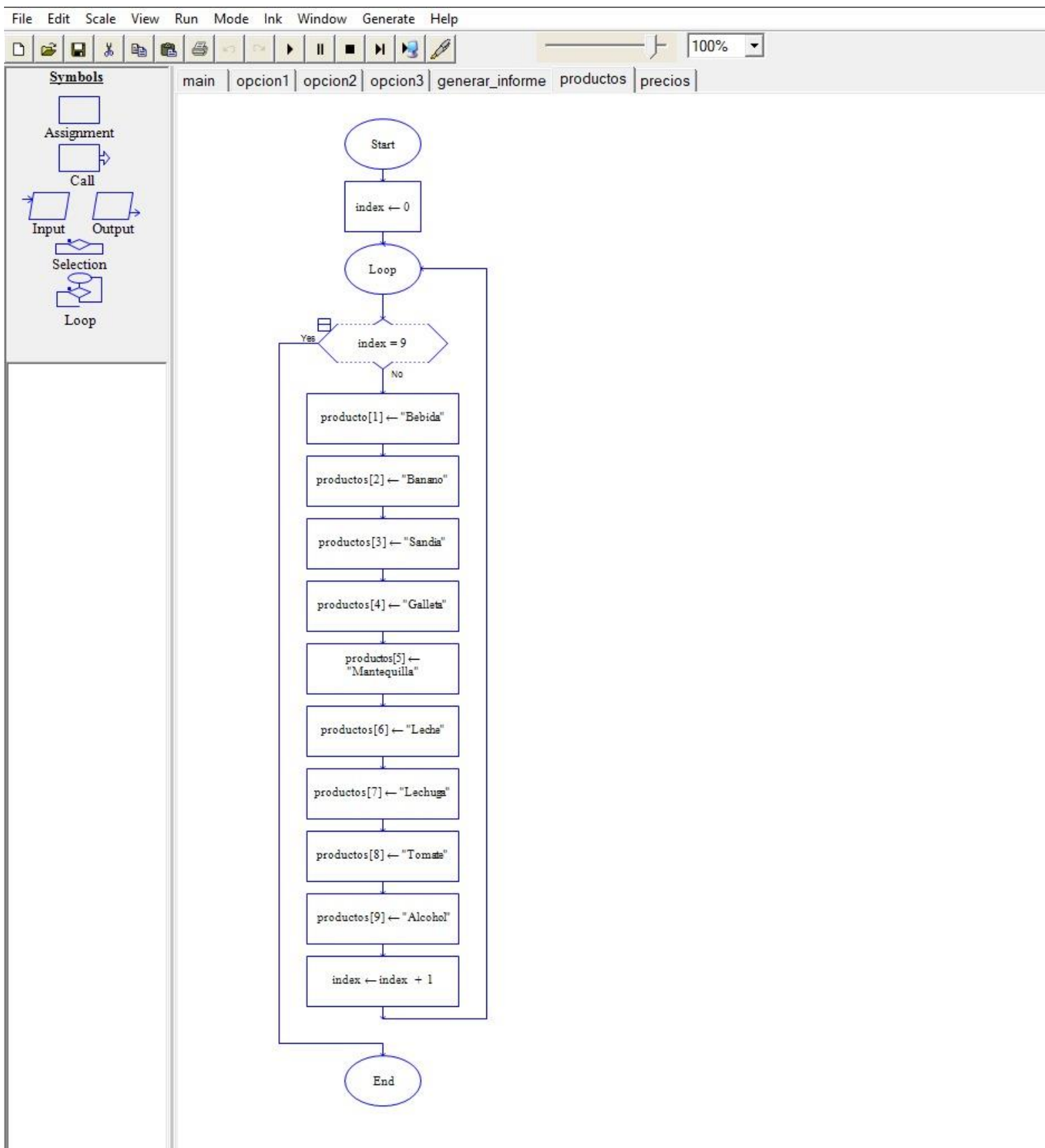
- Generar_informe:



- Precios:



- Productos:



II. Bibliografías

Instituto Nacional de Estadística y Censos. (2021). *Informe de comercios minoristas en Costa Rica*.

<https://www.inec.cr/>

Laudon, K. C., & Laudon, J. P. (2020). *Sistemas de información gerenciales* (14.^a ed.). Pearson Educación.

Pressman, R. S., & Maxim, B. R. (2020). *Ingeniería de software: Un enfoque práctico* (8.^a ed.). McGraw-Hill Education.

Sommerville, I. (2016). *Ingeniería de software* (10.^a ed.). Pearson Educación.

Python.org

<https://docs.python.org/3/>

Stack Overflow

<https://stackoverflow.com/questions/tagged/python>

Automate the Boring Stuff with Python – Al Sweigart

<https://automatetheboringstuff.com/>